CHALLENGE DATA ENGINEER ARTEFACT

Comprendre toute la chaîne technique

De Streamlit aux tables SQL, puis de Bronze à Gold

Objectif : expliquer chaque terme avec des mots simples et suivre une donnée de sa création jusqu'à son interrogation dans Trino.

1. La plateforme en une image mentale

> **Streamlit fabrique un fichier -> MinIO le conserve -> Spark le lit et le contrôle -> Iceberg organise les données en tables -> Trino permet de faire du SQL.**

Votre compréhension est presque exacte, avec une précision importante : Iceberg n'est pas à lui seul le catalogue. Iceberg est le format de table. Un catalogue Iceberg enregistre où se trouvent les tables et leurs métadonnées. Trino et Spark consultent ce catalogue pour retrouver la même table.

Analogie	Composant
Formulaire de commande	Streamlit
Entrepôt de cartons	MinIO
Atelier de contrôle	Spark
Méthode de rangement et inventaire de chaque rayon	Iceberg
Registre indiquant où sont les rayons	Catalogue Iceberg
Guichet de consultation en langage SQL	Trino
Bouton qui démarre tous les bâtiments	Docker Compose

2. Qu'est-ce qu'une donnée raw ?

Raw signifie brut. Une donnée raw est proche de ce que le système source a produit, avant les transformations métier importantes.

Dans le challenge

- La vraie entreprise aurait des core banking systems, API mobile money et logiciels d'assurance.
- Nous ne possédons pas ces systèmes. Streamlit et le générateur les simulent.
- Les CSV produits jouent donc le rôle d'extractions brutes de ces systèmes.

CSV

CSV signifie Comma-Separated Values. C'est un fichier texte tabulaire : la première ligne contient souvent les noms de colonnes, puis chaque ligne représente un enregistrement. Il est simple mais ne garantit ni les types ni les relations.

> **Un CSV peut contenir 'abc' dans une colonne censée contenir un montant. C'est pourquoi Spark doit appliquer un schéma explicite et des validations.**

Fichier, ligne et colonne

- Le fichier regroupe un lot de données.
- Une ligne représente un client, un compte ou une opération.
- Une colonne représente un attribut comme country_code, amount ou status.

3. Streamlit et le générateur

Streamlit est une bibliothèque Python qui permet de créer rapidement une interface web. Ici, elle présente des champs de sélection et des boutons; elle n'est pas le moteur principal de traitement.

Séparation recommandée

Partie	Responsabilité
Interface Streamlit	Récupère les choix de l'utilisateur et affiche l'état.
Moteur de génération Python	Crée réellement les données selon les schémas et règles.
Client MinIO	Envoie le fichier produit dans le bon bucket et le bon chemin.

Pourquoi séparer ?

Le moteur peut être testé sans ouvrir le navigateur, appelé par Streamlit aujourd'hui et réutilisé demain par un autre processus. L'interface reste légère et compréhensible.

Modes

- One-time : un clic crée immédiatement un lot, l'envoie, puis s'arrête.
- Continu : le système crée périodiquement de nouveaux lots toutes les 10 à 60 secondes.

Le mode continu simule l'arrivée régulière de données. Au Level 1, il produit encore des micro-fichiers; le vrai streaming Kafka arrive au Level 3.

4. Le code annexe : point de départ, pas solution finale

Le document dit que le script est fourni à titre indicatif et peut être adapté. Il illustre la logique de génération, mais ne couvre pas toute l'application demandée.

Déjà présent	À construire autour
Pays, devises, métiers et probabilités.	Schémas complets et validations.
Génération de clients.	Accounts, branches, products et trois autres flux.
Exemple de transactions bancaires.	Filtrage cohérent des clés par pays et entité.
Dates et montants aléatoires.	Paramètres Streamlit, seed, volumes dev et nominaux.
Écriture CSV locale.	Upload MinIO, nommage, mode continu, logs et erreurs.

Pourquoi le modifier

- Le code choisit des comptes et agences sans montrer un filtrage par pays.
- Les dates sont codées en dur.
- entity_type est fixé à BANK dans l'extrait.
- Il ne garantit pas tous les KPI futurs.

Ce que l'évaluateur jugera

Il jugera le résultat, la cohérence, la qualité du code et votre capacité à justifier les adaptations. Réutiliser intelligemment le squelette est raisonnable; le copier sans corriger ses limites ne suffit pas.

5. MinIO : stockage objet

Le stockage objet conserve des fichiers appelés objets. Contrairement à un disque classique, on les adresse par un bucket et une clé ressemblant à un chemin.

Terme	Explication simple
Bucket	Grand conteneur logique, comparable à un disque ou une armoire.
Objet	Fichier stocké : CSV, fichier Parquet ou métadonnée Iceberg.
Clé	Nom complet de l'objet dans le bucket, par exemple country_code=CI/type=bank/file.csv.
S3	Protocole/API populaire de stockage objet; MinIO est compatible S3.

Les trois buckets

- raw-landing reçoit les nouveaux CSV.
- lakehouse contient les fichiers de données et métadonnées Iceberg.
- archive contient les fichiers dont le traitement a réussi.

MinIO est-il le lakehouse ?

MinIO fournit la couche de stockage. Le lakehouse apparaît lorsque ce stockage est organisé avec un format de table comme Iceberg, un catalogue et des moteurs comme Spark et Trino.

6. Spark : moteur de calcul distribué

Spark est un moteur capable de traiter beaucoup de données. Distribué signifie qu'il peut répartir le travail sur plusieurs machines ou processus. Dans le challenge local, le volume peut rester modeste, mais la logique est scalable.

Ce que fait un job Spark

- Lit un ou plusieurs objets CSV depuis MinIO.
- Applique un schéma : STRING, DATE, TIMESTAMP, FLOAT, INTEGER, BOOLEAN.
- Vérifie les valeurs obligatoires et les règles métier.
- Joint des tables lorsque nécessaire.
- Sépare les lignes acceptées et rejetées.
- Écrit les lignes acceptées dans une table Iceberg.

Job

Un job est une exécution complète de traitement. Par exemple : charger tous les fichiers bank_txn de Côte d'Ivoire du 10 janvier.

PySpark

PySpark est l'interface Python de Spark. Le code est écrit en Python, mais les opérations sont exécutées par le moteur Spark.

> **Spark transforme les données. Il ne les stocke pas durablement lui-même : il lit et écrit vers MinIO/Iceberg.**

7. Schéma et validation

Terme	Sens
Schéma	Liste officielle des colonnes, de leurs types et parfois de leur caractère obligatoire.
Cast	Conversion d'une valeur vers un type, par exemple le texte '1250.5' vers FLOAT.
NULL	Valeur absente ou inconnue; ce n'est ni zéro ni chaîne vide.
Validation technique	Le timestamp peut-il être lu ? Le montant est-il numérique ?
Validation métier	La devise correspond-elle au pays ? Le compte est-il de type LOAN ?

Que faire d'une mauvaise ligne ?

Elle ne doit pas disparaître silencieusement. Il faut enregistrer la ligne ou son identifiant, le fichier source, la date d'ingestion et la raison du rejet. Ainsi, on peut diagnostiquer et corriger.

Schema inference

L'inférence laisse Spark deviner les types à partir des valeurs observées. Elle est pratique pour explorer, mais risquée en production : une colonne peut changer de type selon le fichier. Le challenge exige un schéma explicite.

8. Iceberg : un format de table sur des fichiers

Un simple dossier de CSV ou Parquet ne sait pas facilement quelles données appartiennent à la table, quelle version est valide ou comment appliquer une modification de façon sûre. Iceberg ajoute cette organisation.

Iceberg apporte	Explication simple
Métadonnées	Liste structurée des fichiers qui composent la table.
Snapshots	Versions successives de la table après chaque écriture.
Transactions	Une écriture apparaît entièrement ou pas du tout.
Évolution de schéma	Possibilité encadrée d'ajouter ou modifier des colonnes.
Partitionnement	Organisation permettant de lire moins de fichiers pour un filtre.
MERGE	Mise à jour/insertion utile pour l'idempotence.

Parquet

Iceberg stocke généralement les données dans des fichiers Parquet. Parquet est un format colonne efficace pour l'analytique. Iceberg organise ces fichiers; Parquet contient les valeurs.

Iceberg n'est ni un serveur ni un moteur SQL. C'est un standard de table partagé par Spark et Trino.

9. Le catalogue Iceberg

Le catalogue est le registre central qui associe un nom logique comme bronze.bank_transactions à l'emplacement et aux métadonnées de la table.

Pourquoi est-il nécessaire ?

Spark doit savoir où écrire. Trino doit retrouver exactement la même table. Sans registre partagé, chaque moteur pourrait avoir une vision différente.

Élément	Exemple
Catalogue	lakehouse
Schéma ou namespace	raw, bronze, silver ou gold
Table	bank_transactions
Nom complet	lakehouse.bronze.bank_transactions

Choix possibles

L'énoncé autorise notamment un Iceberg REST Catalog ou un Hive Metastore. Le choix doit être compatible avec Spark et Trino, conteneurisé et documenté.

Correction de la formulation

Spark n'écrit pas 'dans Iceberg pour le catalogue'. Spark écrit une table au format Iceberg sur MinIO et enregistre/actualise cette table dans le catalogue. Trino consulte ensuite le catalogue et lit les fichiers Iceberg sur MinIO.

10. Trino : moteur SQL de consultation

Trino reçoit une requête SQL, demande au catalogue où se trouve la table, lit les fichiers nécessaires dans MinIO et calcule le résultat.

SQL

SQL est le langage utilisé pour interroger les tables : sélectionner des colonnes, filtrer un pays, grouper par date ou additionner des montants.

Étape d'une requête	Ce qui se passe
1. SELECT	L'utilisateur demande un résultat.
2. Catalogue	Trino résout le nom de la table.
3. Métadonnées Iceberg	Trino identifie les fichiers et partitions utiles.
4. MinIO	Trino lit les données nécessaires.
5. Calcul	Trino filtre, joint ou agrège.
6. Résultat	Les lignes sont retournées à l'utilisateur.

Trino ne transforme pas durablement les données du pipeline dans ce challenge. Son rôle principal est de les exposer et de les interroger.

11. Partition et idempotence

Partition

Partitionner consiste à organiser logiquement les données selon des colonnes souvent filtrées, ici country_code et date. Une requête sur CI et un jour précis peut éviter de lire tous les pays et toutes les dates.

Idempotence

Une opération est idempotente si la refaire produit le même état final. Traiter deux fois le même fichier ne doit pas créer deux fois la même transaction.

Stratégie	Rôle
Identifiant métier unique	Repérer transaction_id, operation_id, payment_id ou repayment_id déjà présent.
source_file	Tracer le fichier ayant produit la ligne.
MERGE	Insérer uniquement les identifiants absents ou actualiser selon la règle.
Journal de fichiers	Savoir quels fichiers ont déjà réussi.
Archive après succès	Éviter qu'un fichier encore actif disparaisse avant confirmation.

Le contrôle par identifiant protège contre les doublons dans plusieurs fichiers. Le contrôle par fichier protège contre la relance exacte d'un même lot. Les deux sont complémentaires.

12. Docker et Docker Compose

Un conteneur empaquette une application avec son environnement d'exécution. Une image est le modèle immuable; un conteneur est une instance lancée de cette image.

Terme	Explication
Dockerfile	Recette pour construire une image.
Image	Paquet versionné de l'application et de ses dépendances.
Conteneur	Processus isolé lancé depuis une image.
Volume	Stockage persistant survivant au redémarrage du conteneur.
Réseau Docker	Permet aux services de communiquer par leur nom.
Docker Compose	Fichier déclarant plusieurs services à lancer ensemble.

Un seul docker compose up

Cela signifie que l'évaluateur ne doit pas démarrer manuellement MinIO, puis Spark, puis Trino. Compose crée le réseau, les volumes, les variables et lance la stack. Des initialisations peuvent rester nécessaires, mais elles doivent être automatisées ou clairement documentées.

13. Raw, Bronze, Silver et Gold

Zone	Ce qu'elle contient	Transformation
raw-landing	CSV exactement déposés.	Aucune; zone d'arrivée.
Bronze	Tables Iceberg proches des sources avec traçabilité.	Schéma et validation minimale.
Silver	Données fiables et harmonisées.	Déduplication, jointures, nulls, EUR, règles métier.
Gold	Tables orientées décision.	Agrégations et KPI.

Le chevauchement raw.* / bronze.*

Le Level 1 demande des tables raw.*; le Level 2 demande bronze.*. Une architecture claire peut considérer raw.* comme première implémentation du Level 1 puis faire de bronze.* la couche pérenne du Level 2, ou conserver les deux avec des responsabilités distinctes. Le README doit expliquer ce choix.

Conversion EUR

Silver ajoute un montant comparable, par exemple amount_eur, sans nécessairement supprimer amount et currency d'origine. Le taux doit être identifié, daté et reproductible.

14. Airflow : orchestrer, pas calculer

Airflow gère l'ordre et l'état des tâches. Un DAG est un graphe orienté sans boucle décrivant les dépendances entre tâches.

Terme	Explication simple
DAG	Plan complet d'un workflow.
Task	Une étape : lancer un job Spark, contrôler ou publier.
Dependency	Règle indiquant qu'une tâche attend une autre.
Schedule	Moment automatique d'exécution.
Retry	Nouvel essai après une erreur temporaire.
Backfill	Rejeu contrôlé d'une période historique.
Connection	Configuration d'accès à un service sans écrire le secret dans le DAG.

dag_ingest_raw -> dag_bronze_to_silver -> dag_silver_to_gold. Le reporting réglementaire s'exécute ensuite quotidiennement selon sa planification.

Paramétrable par pays

Le DAG reçoit country_code et éventuellement une période. On peut ainsi retraiter seulement le Sénégal sans recalculer les huit pays.

15. Voyage complet d'une transaction

Étape	État de la transaction
1. Streamlit	L'utilisateur choisit banque, CI, période et volume.
2. Générateur	Crée un UUID, choisit des comptes/agence valides, montant XOF et statut.
3. CSV	Écrit la ligne dans bank_txn_CI_YYYYMMDD_NN.csv.
4. MinIO	Dépose le fichier dans raw-landing sous un chemin CI/bank.
5. Spark	Lit, caste, valide les clés, la devise et les champs.
6. Iceberg Bronze	Écrit la ligne avec ingestion_timestamp et source_file.
7. Archive	Déplace le CSV après succès.
8. Silver	Déduplique, enrichit et calcule amount_eur.
9. Gold	Agrège la ligne dans le volume journalier et éventuellement l'ARPU.
10. Trino	Retourne le KPI dans une requête SQL filtrée sur CI.

C'est cette histoire complète qu'il faudra montrer dans la démonstration et expliquer en entretien.

16. Ce qui doit être testé

Niveau	Exemples de tests
Générateur	Colonnes, distributions, devises, clés existantes, règles conditionnelles.
MinIO	Bucket présent, chemin correct, upload et lecture.
Spark	Bonnes lignes acceptées, mauvaises lignes rejetées avec raison.
Iceberg	Tables créées, schéma et partitions corrects, relance sans doublon.
Trino	Huit tables accessibles, sommes et comptes plausibles.
Airflow	Dépendances, retries, paramètres pays, échec visible et backfill.
Gold	Formules, grains, filtres, valeurs attendues sur un petit jeu connu.

Test unitaire et test de bout en bout

- Un test unitaire vérifie une petite fonction isolée, par exemple country -> currency.
- Un test d'intégration vérifie que deux composants communiquent, par exemple Spark vers MinIO.
- Un test end-to-end suit un fichier depuis la génération jusqu'à la requête Trino.

17. Résumé technique à mémoriser

MinIO stocke. Spark calcule et écrit. Iceberg structure les tables sur MinIO. Le catalogue référence ces tables. Trino les interroge. Airflow ordonne les jobs. Docker Compose lance l'ensemble.

Phrase de restitution possible

L'application Streamlit simule les systèmes sources et génère des lots CSV cohérents. Elle les dépose dans la zone raw-landing de MinIO. Des jobs PySpark appliquent des schémas et des contrôles, puis écrivent des tables au format Iceberg sur le bucket lakehouse. Un catalogue partagé permet à Spark et Trino d'identifier les mêmes tables. Trino les expose en SQL. Au Level 2, Airflow orchestre le passage Bronze, Silver et Gold et les tables Gold fournissent les KPI métier.

Nuances importantes

- MinIO seul n'est pas toute la plateforme lakehouse.
- Iceberg n'est pas le catalogue, même s'il s'appuie sur un catalogue.
- Trino ne remplace pas Spark pour les pipelines demandés.
- Airflow ne transporte pas lui-même les données; il orchestre les traitements.
- Le script fourni est modifiable et doit être complété.

Si vous pouvez expliquer le voyage complet d'une ligne et la responsabilité de chaque composant, vous maîtrisez l'architecture technique des Levels 1 et 2.